Natural Language Processing
Laboratorio #8
Embedding

Instrucciones generales

- Individual.
- Entrega: Viernes 23 de octubre, 2026. 23:59.

Datos

Este laboratorio reutiliza los dos corpus:

Recurso	Uso en el laboratorio	Origen
Spanish News Classification	Corpus principal: entrenar embeddings y clasificar noticias	https://www.kaggle.com/datasets/kevinmorgado/spanish-news-classification
Don Quijote de la Mancha	Segundo corpus: comparar cómo cambia el significado según el texto	https://www.kaggle.com/datasets/manuelmaaf97/quijote
fastText SUC, tamaño M (100 dimensiones)	Vectores preentrenados en español (2,600 millones de palabras)	https://github.com/dccuchile/spanish-word-embeddings/blob/master/emb-from-suc.md

El archivo preentrenado pesa 1.1 GB. Cárguelo una sola vez y limite el vocabulario a las palabras más frecuentes:

```
from gensim.models import KeyedVectors
suc = KeyedVectors.load_word2vec_format("embeddings-m-model.vec",
limit=200_000)
```

Si su computadora tiene poca memoria, use `limit=100_000` o trabaje en Google Colab.

1. Preparación de los corpus

word2vec aprende de ventanas de palabras vecinas, así que el corpus debe prepararse como una lista de oraciones, cada una con sus tokens en orden (texto → tokens → IDs → vectores).

- Cargue el corpus de noticias y el Quijote. Segmente cada uno en oraciones y tokenice cada oración en palabras, conservando el orden. El resultado debe ser una lista de listas de tokens, que es el formato que espera gensim.

- Defina y justifique su normalización: minúsculas, puntuación, números, tildes. Retome el pipeline del Laboratorio 1: ¿vale la pena quitar stopwords o lematizar antes de entrenar embeddings? Piense en qué le pasa a la ventana de contexto y en qué se pierde al convertir "gobiernos" y "gobernó" en una sola forma.
- Revise si el vocabulario del modelo fastText SUC distingue mayúsculas y tildes (por ejemplo, busque "españa", "España" y "espana"). Su normalización debe ser compatible con la de SUC para que la comparación de las secciones 5 y 8 sea justa.
- Reporte, en una tabla con una fila por corpus: número de documentos, oraciones, tokens y tipos.
- Construya el vocabulario con tres valores de frecuencia mínima (`min_count` = 1, 2 y 5) y reporte para cada uno el tamaño del vocabulario y el porcentaje de tokens que quedan fuera.
- Tome una oración del corpus de noticias y genere a mano, con código propio, todos los pares (palabra central, contexto) de skip-gram con ventana 2, como en la diapositiva de la clase.
- Calcule cuántos pares de entrenamiento produce todo el corpus de noticias con ventana 2 y con ventana 5. Compárelo con el número de tokens: ¿por qué word2vec no necesita datos etiquetados?

2. Co-ocurrencia, PPMI y SVD

Antes de entrenar una red, aplique la hipótesis distribucional de la forma más directa: contando contextos. Trabaje con el corpus de noticias.

- Construya la matriz de co-ocurrencia palabra × palabra con ventana ±2 para las 2,000 palabras más frecuentes. Reporte su forma y su porcentaje de ceros, y compárelo con la dispersión de la matriz BoW del Laboratorio 2.
- Elija 5 pares de palabras (relacionadas y no relacionadas) y calcule la similitud coseno entre sus filas. ¿Coincide con su intuición?
- Liste las 10 palabras con más co-ocurrencias con "empresa" y con "gobierno". ¿Qué tipo de palabras dominan la lista?
- Transforme la matriz con PPMI e implemente usted la fórmula con NumPy:

$$PPMI(w, c) = \max(0, \log_2 [P(w, c) / (P(w) \cdot P(c))])$$

- Repita las 10 palabras más asociadas a "empresa" y "gobierno" con PPMI. ¿Qué cambió y por qué?
- Reduzca la matriz PPMI a 100 dimensiones con `TruncatedSVD` de scikit-learn. Estos vectores densos (estilo LSA) son su primer modelo de embeddings; guárdelos como `KeyedVectors` para usarlos en la sección 5.
- Explique con sus propias palabras por qué las filas de la matriz ya son vectores de palabras y qué problemas resuelven PPMI y SVD.

3. Investigación

Responda brevemente, con sus propias palabras (máximo media página en el reporte):

- ¿En qué se diferencian CBOW y skip-gram? ¿Cuál se espera que funcione mejor con palabras poco frecuentes?
- ¿Qué problema resuelve negative sampling frente a un softmax sobre todo el vocabulario? Escriba la probabilidad σ(v_w · c_c) y explique qué representa cada vector.
- ¿Por qué word2vec aprende dos tablas de vectores (palabra y contexto) y cuál se conserva al final?
- ¿En qué se diferencian word2vec (predictivo y local), GloVe (conteos globales) y fastText (subpalabras)? ¿Por qué fastText es especialmente útil en español?
- Describa el propósito de estos parámetros de `gensim.models.Word2Vec`: `vector_size`, `window`, `min_count`, `sg`, `negative`, `sample` y `epochs`.
- Describa `nn.Embedding` (`num_embeddings`, `embedding_dim`, `padding_idx`), `nn.Embedding.from_pretrained` (`freeze`) y `nn.EmbeddingBag` (`mode`), que usará en la sección 8. ¿Por qué multiplicar un vector one-hot por la matriz E equivale a seleccionar una fila?

4. Entrenar Word2Vec y fastText

Entrene sus propios embeddings con gensim sobre los corpus de la sección 1.

- Entrene un modelo base `Word2Vec` sobre el corpus de noticias con `vector_size=100`, `window=5`, `min_count=2`, `sg=1`, `negative=5` y al menos 10 épocas.
- Entrene al menos 4 variantes más cambiando un parámetro a la vez: CBOW frente a skip-gram (`sg`), ventana 2 frente a 10 (`window`) y dimensión 50 frente a 300 (`vector_size`).
- Defina un conjunto fijo de 6 palabras de control relacionadas con las noticias (por ejemplo: “banco”, “gobierno”, “empresa”, “mercado”, “tecnología” y “sostenibilidad”; ajuste la lista si alguna no aparece en su vocabulario).
- Para cada variante, registre en una tabla la configuración, el tiempo de entrenamiento y las 5 palabras más cercanas a cada palabra de control. ¿Las ventanas pequeñas dan vecinos más sintácticos (otras formas de la misma palabra) y las grandes, más temáticos?
- Entrene un modelo con la misma configuración base sobre el Quijote. Para 5 palabras presentes en ambos corpus (por ejemplo “señor”, “dinero”, “casa”, “caballo”, “palabra”), compare los vecinos de ambos modelos. ¿Qué dice esto sobre la frase “el significado viene del corpus”?
- Entrene un modelo `FastText` de gensim sobre el corpus de noticias con la configuración base. Obtenga vectores y vecinos para 3 palabras que no estén en el vocabulario (una con error de dedo, una conjugación poco común y una palabra inventada como “criptoempresa”). Intente lo mismo con Word2Vec y explique la diferencia.
- Elija su mejor modelo propio para el resto del laboratorio y justifique la elección con lo que observó en los vecinos.

5. Similitud, vecinos y analogías

Compare cuatro modelos: SVD sobre PPMI (sección 2), su mejor Word2Vec de noticias, el Word2Vec del Quijote y fastText SUC.

5.1 Similitud coseno y vecinos

- Implemente su propia función `coseno(u, v)` con NumPy y verifique que da el mismo valor que `similarity` de gensim en 3 pares.
- Para 5 palabras de su elección, muestre en una tabla las 5 vecinas más cercanas en cada uno de los cuatro modelos.
- Calcule la similitud de al menos 4 pares de antónimos ("bueno" / "malo", "subir" / "bajar", "ganar" / "perder", "público" / "privado"). ¿Son altas o bajas? Explique por qué similitud distribucional no es lo mismo que sinonimia.

5.2 Conjunto de similitud propio

- Construya una lista de 20 pares de palabras y asígneles usted mismo un puntaje de similitud de 0 a 10, como hizo con el gold standard del Laboratorio 6. Incluya pares muy parecidos, relacionados y sin relación.
- Calcule la correlación de Spearman (`scipy.stats.spearmanr`) entre sus puntajes y el coseno de cada modelo. ¿Qué modelo se parece más a su juicio? Reporte cuántos pares quedaron fuera del vocabulario de cada uno.

5.3 Analogías

- Implemente su propia función `analogia(a, b, c, k=5)` que resuelva "a es a b como c es a ?" con el vector b − a + c, similitud coseno sobre vectores normalizados y excluyendo a, b y c del resultado. Verifique que coincide con `most_similar(positive=[b, c], negative=[a])` de gensim.
- Escriba al menos 15 analogías propias en español de al menos 5 tipos: género (hombre : rey :: mujer : reina), país–capital, país–gentilicio, singular–plural, tiempos verbales (comía : comerá), masculino–femenino de adjetivos.
- Para cada analogía y modelo, reporte el top 5 y la posición de la respuesta correcta. Resuma el porcentaje de aciertos en top 1 y top 5 por tipo de analogía en una tabla.
- Repita 5 analogías sin excluir las palabras de la consulta. ¿Qué palabra aparece en primer lugar? ¿Qué dice esto sobre el "≈" de rey − hombre + mujer ≈ reina?

6. Visualización del espacio

- Elija alrededor de 150 palabras de 5 o 6 grupos temáticos (países, meses, números, verbos de movimiento, términos económicos, profesiones) presentes en su mejor modelo y en fastText SUC.
- Proyéctelas a 2 dimensiones con PCA y con t-SNE (`sklearn.manifold.TSNE`), coloreando cada punto por grupo y rotulando algunas palabras.
- ¿Se forman los grupos esperados? ¿En qué modelo se separan mejor?
- Compare PCA y t-SNE: ¿por qué en t-SNE se puede confiar en los grupos pero no en la distancia entre grupos?

7. Análisis

Responda con base en sus resultados:

- ¿Se parecen los vecinos del modelo de conteos (SVD sobre PPMI) a los de word2vec? ¿Qué ventaja tiene uno sobre otro? Relacione su respuesta con la idea de que GloVe une ambos enfoques.
- ¿Qué tan lejos quedó su Word2Vec de fastText SUC en vecinos, similitud y analogías? ¿Cuánto de la diferencia atribuye al tamaño del corpus (cientos de miles frente a miles de millones de palabras) y cuánto al método?
- ¿En qué tipos de analogía funcionó mejor la aritmética vectorial y en cuáles falló? ¿Es el "≈" una igualdad?
- Compare los vecinos del modelo de noticias y del Quijote. Igual que con el tagger del Laboratorio 6, ¿qué problemas tendría usar embeddings entrenados en un dominio o una época para otro?
- En la clasificación de noticias, ¿los embeddings superaron a TF-IDF? ¿Cuándo convino congelar la tabla y cuándo hacer fine-tuning? ¿Cómo cambia la respuesta con 10 % de los datos?
- La palabra "banco" tiene un solo vector en todos sus modelos. Busque sus vecinos y explique qué sentidos se mezclan. ¿Cómo resuelven este problema los embeddings contextuales como BERT, que verá en las próximas semanas?
- Retomando la sección 7: si tuviera que usar embeddings preentrenados en un sistema que toma decisiones sobre personas, ¿qué auditaría antes de ponerlo en producción?

Entregables

Entregable	Contenido
Link a repositorio	Repositorio (por ejemplo GitHub) con el notebook comentado y el código versionado del laboratorio, un requirements.txt con las versiones de gensim, scikit-learn y PyTorch, y los vectores de su mejor modelo (.kv o .txt) o un enlace para descargarlos. No suba el archivo de fastText SUC.
Reporte escrito	Decisiones de preprocesamiento, investigación (sección 3), tablas de variantes y vecinos, resultados de similitud y analogías, visualizaciones, análisis de sesgos, resultados de clasificación y búsqueda, y respuestas de la sección 9.